UNIVERSIDADE DE MOGI DAS CRUZES

MARIANA DA ROCHA PEREIRA MOREIRA

PEDIU PATRÃO:

Dashboard administrativo de vendas e pedidos

Mogi das Cruzes, SP
2026

SUMÁRIO

1 VISÃO GERAL **3**

1.1 Tecnologias 3

1.2 Arquitetura 3

2 REQUISITOS FUNCIONAIS **4**

2.1 Controle de acesso 5

3 REQUISITOS NÃO FUNCIONAIS **5**

4 REGRAS DE CÁLCULO DOS INDICADORES **6**

5 PRINCIPAIS DECISÕES DE ANÁLISE E PROJETO **7**

6 DOCUMENTAÇÃO DA API **8**

6.1 Informações gerais 8

6.2 Filtros comuns a todos os endpoints (RF05) 8

6.3 Formato da resposta de sucesso (200 OK) 9

6.4 Formato e códigos de erro 9

6.5 Endpoints 9

6.5.1 GET /api/dashboard/indicadores (RF01) 9

6.5.2 GET /api/dashboard/vendas (RF02) 10

6.5.3 GET /api/dashboard/vendas-por-hora (RF02) 10

6.5.4 GET /api/dashboard/vendas-por-tipo (RF02) 11

6.5.5 GET /api/dashboard/status (RF03) 11

6.5.6 GET /api/dashboard/produtos-mais-vendidos (RF04) 11

6.5.7 GET /api/dashboard/pedidos-recentes (apoio) 12

6.6 Exemplos de erro 12

7 DADOS DE DEMONSTRAÇÃO **13**

8 EVIDÊNCIAS DE TESTES **14**

8.1 Testes automatizados 14

8.1.1 DashboardServiceTest: cálculos (MongoDB simulado com Mockito; “hoje” fixado em 02/11/2026) 14

8.1.2 DashboardApiTest: camada web com o SecurityConfig real 15

8.2 Testes manuais da tela e da API 15

8.3 Imagens 17

9 GITFLOW E VERSIONAMENTO **22**
10 COMO EXECUTAR **22**

1. VISÃO GERAL

O Dashboard Administrativo dá ao ADMIN e ao GERENTE uma visão consolidada das vendas e da operação da pizzaria: faturamento, volume de pedidos, ticket médio, evolução das vendas, horários de pico, situação dos pedidos e produtos mais vendidos, com filtros por período, tipo de atendimento, status e responsável.

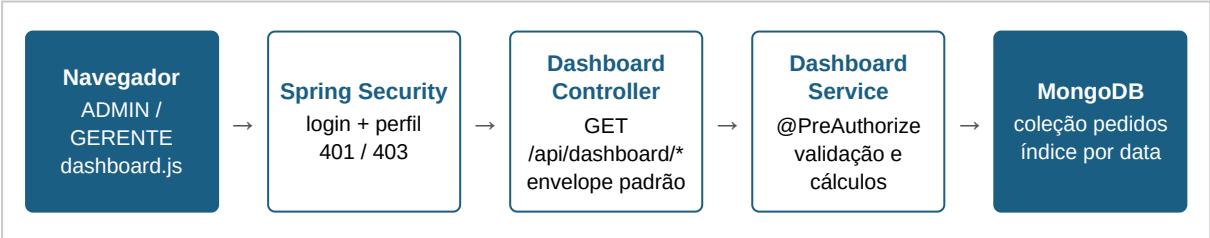
A solução tem duas partes:

- uma API REST própria (`/api/dashboard/**`), somente leitura, que consulta o MongoDB e calcula os indicadores no servidor;
- uma tela (`/dashboard`) que consome essa API via JavaScript e apresenta os dados em cards, gráficos e tabelas.

1.1 TECNOLOGIAS

Camada	Tecnologia
Back-end	Java 21, Spring Boot 3.4.4, Spring Web, Spring Security
Banco de dados	MongoDB (Spring Data MongoDB / `MongoTemplate`)
Front-end	Thymeleaf, Bootstrap 5, Bootstrap Icons, JavaScript (Fetch API)
Gráficos	Chart.js 4.4.1
Testes	JUnit 5, Mockito, Spring MockMvc, Spring Security Test
Versionamento	Git e GitHub, fluxo Gitflow

1.2 ARQUITETURA



Componente	Responsabilidade
`DashboardViewController`	Entrega a página `/dashboard` com as opções dos filtros
`index.html`, `dashboard.js`, `dashboard.css`	Tela: busca os dados na API, desenha os gráficos, trata erros e atualização
`DashboardController`	Endpoints REST `/api/dashboard/**` e envelope padrão da resposta
`DashboardService`	Validação dos filtros, consulta ao MongoDB e todos os cálculos

Componente	Responsabilidade
`dto/dashboard/*`	Records imutáveis de entrada (filtro) e de saída (respostas)
`ApiExceptionHandler`	Converte exceções em respostas JSON 400, 404 e 422
`SecurityConfig`	Autenticação, autorização por perfil (401 e 403) e CSRF
`MongoInitConfig`	Cria o índice `idx_pedidos_dataHoraEntrada`
`DemoDataInitializer`	Gera 180 pedidos fictícios realistas para demonstração

2. REQUISITOS FUNCIONAIS

ID	Requisito	Como foi atendido	Onde
RF01	Indicadores gerais de vendas	Faturamento, total de pedidos, ticket médio, itens vendidos, descontos concedidos, cancelamentos e taxa de cancelamento, com variação percentual em relação ao período anterior de mesma duração	`/indicadores` cards do topo
RF02	Análise de vendas (gráficos)	Evolução das vendas por dia, semana ou mês (faturamento e nº de pedidos), pedidos por hora do dia com destaque do horário de pico, e entrega x retirada	`/vendas`, `/vendas-por-hora`, `/vendas-por-tipo`
RF03	Situação dos pedidos	Quantidade e percentual por status, com as mesmas cores do sistema; pedidos em andamento, finalizados e cancelados; tempos médios de preparo, até a saída e total	`/status`
RF04	Produtos mais vendidos	Ranking ordenável por quantidade ou faturamento, com nº de pedidos e participação no faturamento	`/produtos-mais-vendidos`
RF05	Filtros de consulta	Data inicial e final, tipo de atendimento, status e responsável, mais os atalhos Hoje / 7 / 30 / 60 dias. Todos os cards, gráficos e tabelas usam os mesmos filtros. Padrão: últimos 30 dias. O período exibido e os filtros ativos ficam em destaque na tela	Parâmetros de todos os endpoints barra de filtros
RF06	Integração com a API	A tela não acessa o banco: todos os dados vêm da API REST via `fetch`, com as 7 chamadas em paralelo. Os endpoints são integrados ao MongoDB	`dashboard.js` → `/api/dashboard/**`

ID	Requisito	Como foi atendido	Onde
RF07	Atualização e tratamento de dados	Atualização automática a cada 60 s (pode ser desligada e pausa com a aba oculta), botão Atualizar, horário da última atualização, indicador de carregamento, aviso de “sem dados” e mensagens específicas para 400/422, 401, 403 e falha de conexão, mantendo os últimos dados válidos na tela	Tela e `ApiExceptionHandler`

2.1 CONTROLE DE ACESSO

Perfil	Tela /dashboard	API /api/dashboard/**
ADMIN	Acessa	200
GERENTE	Acessa	200
ATENDENTE	Página de acesso negado (link oculto no menu)	403
Não autenticado	Redireciona para o login	401

3. REQUISITOS NÃO FUNCIONAIS

ID	Categoria	Requisito	Como foi atendido
RNF01	Segurança	Somente ADMIN e GERENTE acessam o dashboard	Duas camadas: regra de URL no `SecurityConfig` e `@PreAuthorize` no `DashboardService`. Esconder o link do menu é só usabilidade
RNF02	Segurança	Proteção contra injeção de script (XSS)	Textos vindos do banco, como o nome do cliente, entram na tela com `textContent`, nunca como HTML
RNF03	Segurança	API somente leitura	Apenas `GET`; nenhum endpoint altera dados. O CSRF continua ativo no sistema
RNF04	Desempenho	Consultas eficientes	O período é filtrado no MongoDB, sobre o índice `idx_pedidos_dataHoraEntrada`
RNF05	Desempenho	Proteção contra consultas excessivas	Período máximo de 366 dias e limite de 1 a 50 itens nas listas
RNF06	Desempenho	Carregamento rápido da tela	7 requisições em paralelo (`Promise.all`); os gráficos são atualizados, não recriados

ID	Categoria	Requisito	Como foi atendido
RNF07	Confiabilidade	Precisão dos valores monetários	`BigDecimal` com arredondamento `HALF_UP` e 2 casas; nada de `double` para dinheiro
RNF08	Confiabilidade	Consistência das regras	Cancelados não entram em faturamento, itens nem gráficos de vendas, mas entram na taxa de cancelamento
RNF09	Disponibilidade	Tolerância a falhas	Em falha de rede ou servidor, a tela avisa e mantém os últimos dados válidos
RNF10	Usabilidade	Interface clara e coerente com o sistema	Cores da marca (amarelo `#ffc107` e grafite `#23272f`), mesmas cores de status das outras telas, valores em R$ no padrão brasileiro, período em destaque
RNF11	Usabilidade	Filtros compartilháveis	Os filtros ficam na URL: recarregar ou enviar o link mantém a consulta
RNF12	Usabilidade	Responsividade	Grid do Bootstrap: cards e gráficos se reorganizam em telas menores
RNF13	Interoperabilidade	Respostas padronizadas	Sucesso no envelope `{ filtros, atualizadoEm, dados }`; erros no formato `{ timestamp, status, erro, mensagem }`
RNF14	Manutenibilidade	Código organizado em camadas	Controller → Service → MongoDB, DTOs como `record`, sem regra de negócio no controller nem na tela
RNF15	Testabilidade	Testes automatizados reprodutíveis	24 testes novos (61 no total); `Clock` injetável fixa a data de “hoje”; banco simulado com Mockito
RNF16	Eficiência	Atualização automática econômica	Intervalo de 60 s, pausado quando a aba do navegador está oculta

4. REGRAS DE CÁLCULO DOS INDICADORES

Indicador	Fórmula
Faturamento	Soma do `valorTotal` (já com desconto) dos pedidos não cancelados
Total de pedidos	Todos os pedidos do período, inclusive os cancelados
Pedidos válidos	Total de pedidos - cancelados
Ticket médio	Faturamento ÷ pedidos válidos (0,00 se não houver pedidos válidos)
Itens vendidos	Soma das quantidades dos itens dos pedidos não cancelados

Indicador	Fórmula
Descontos concedidos	Soma do `valorDesconto` dos pedidos não cancelados
Taxa de cancelamento	Cancelados ÷ total de pedidos × 100
Variação	(atual - anterior) ÷ anterior × 100. O período anterior tem o mesmo nº de dias e termina na véspera do início. Fica `null` quando o anterior é zero
Tempo médio de preparo	Média de (momento em que ficou PRONTO - entrada)
Tempo médio até a saída	Média de (saída para entrega ou retirada - entrada)
Tempo médio total	Média de (finalização - entrada)
Participação do produto	Faturamento bruto do produto ÷ faturamento bruto de todos os itens × 100

O período considera a data de entrada do pedido, de 00:00 da data inicial até 23:59:59 da data final, no fuso `America/Sao_Paulo`. Na série semanal, cada grupo começa na segunda-feira, e períodos sem venda aparecem com zero para não deixar buracos no gráfico.

5. PRINCIPAIS DECISÕES DE ANÁLISE E PROJETO

Decisão	Justificativa
Cálculo no servidor, não na tela	As regras ficam em um lugar só, testado, e o navegador não recebe os pedidos brutos, só os números prontos
Soma em Java com `BigDecimal`	No banco os valores ficam como texto, o que impede o `$sum` do MongoDB. O período é filtrado no banco, com índice, e as somas são feitas em Java com precisão decimal
Um endpoint por assunto	Cada endpoint tem uma responsabilidade clara e é testável sozinho. A tela chama todos em paralelo; trocar “Dia/Semana/Mês” ou “Quantidade/Faturamento” recarrega só o que mudou
Envelope `{ filtros, atualizadoEm, dados }`	A tela sabe exatamente qual período foi aplicado (inclusive o padrão de 30 dias) e quando os dados foram calculados, informações mostradas na faixa de período e em “Última atualização”
422 para regra, 400 para formato	Data final antes da inicial é uma requisição bem formada que viola uma regra (422); `agrupamento=ANO` ou `31/10/2026` nem podem ser interpretados (400). As mensagens dizem os valores aceitos
Comparação com o período anterior	Um número sozinho diz pouco; “+73,5% em relação aos 30 dias anteriores” mostra tendência e apoia a decisão do gerente
Segurança em duas camadas	URL protegida no `SecurityConfig` e método protegido no service: mesmo que alguém chame o service por outro caminho, só ADMIN e GERENTE passam

Decisão	Justificativa
Dados de demonstração realistas	Pedidos fictícios com padrões reais (picos no almoço e no jantar, mais movimento no fim de semana) e semente fixa, para os gráficos fazerem sentido e serem reprodutíveis
Gitflow	Cada funcionalidade em sua `feature/*`, integrada à `develop` com merge `--no-ff` e publicada na `main` por uma `release`

6. DOCUMENTAÇÃO DA API

6.1 INFORMAÇÕES GERAIS

URL base	`http://localhost:8080/api/dashboard`
Métodos	Somente `GET`
Formato	JSON (`application/json`)
Autenticação	Sessão do Spring Security (login em `/login`, cookie `JSESSIONID`)
Autorização	Perfis ADMIN e GERENTE
Datas	Entrada `AAAA-MM-DD` · saída `AAAA-MM-DDTHH:mm:ss`
Valores monetários	Número decimal com 2 casas (ex.: `10929.89`)

6.2 FILTROS COMUNS A TODOS OS ENDPOINTS (RF05)

Parâmetro	Tipo / valores	Padrão	Descrição
`dataInicio`	data `AAAA-MM-DD`	`dataFim` - 29 dias	Início do período (data de entrada do pedido)
`dataFim`	data `AAAA-MM-DD`	hoje	Fim do período (inclusive)
`tipoEntrega`	`ENTREGA` | `RETIRADA`	todos	Tipo de atendimento
`status`	`RECEBIDO` | `EM_PREPARACAO` | `PRONTO` | `SAIU_PARA_ENTREGA` | `RETIRADO` | `FINALIZADO` | `CANCELADO`	todos	Status atual do pedido
`responsavel`	texto	todos	Usuário que registrou o pedido (ex.: `atendente01`)

Todos os filtros são opcionais. Regras: `dataFim` ≥ `dataInicio` e período de no máximo 366 dias.

6.3 FORMATO DA RESPOSTA DE SUCESSO (200 OK)

```
{
  "filtros": { "dataInicio": "2026-10-04", "dataFim": "2026-11-02", "tipoEntrega":
null, "status": null, "responsavel": null },
  "atualizadoEm": "2026-11-02T22:55:23",
  "dados": { ... conteúdo de cada endpoint ... }
}
```

`filtros`	Filtros efetivamente aplicados, já com os valores padrão
`atualizadoEm`	Data e hora em que os dados foram calculados
`dados`	Conteúdo específico de cada endpoint

6.4 FORMATO E CÓDIGOS DE ERRO

```
{
  "timestamp": "2026-11-02T22:56:47.812",
  "status": 422,
  "erro": "Regra de negócio violada",
  "mensagem": "A data final não pode ser anterior à data inicial."
}
```

Status	Quando	erro	Exemplos de mensagem
400	Parâmetro com valor ou formato inválido	Parâmetro inválido	*Parâmetro 'agrupamento' com valor inválido: 'ANO'. Valores aceitos: DIA, SEMANA, MES.*
401	Requisição sem login	(corpo vazio)	
403	Autenticado sem perfil ADMIN ou GERENTE	Acesso negado	*Seu perfil não tem permissão para realizar esta operação.*
422	Regra de negócio violada	Regra de negócio violada	*A data final não pode ser anterior à data inicial. · O período máximo de consulta é de 366 dias. · O limite deve estar entre 1 e 50.*

6.5 ENDPOINTS

Os exemplos de resposta usam os dados de demonstração de 02/11/2026. Nos trechos marcados com “...”, parte da lista foi omitida.

6.5.1 GET /api/dashboard/indicadores (RF01)

Indicadores gerais do período e comparação com o período anterior de mesma duração. Parâmetros: apenas os filtros comuns.

```
GET /api/dashboard/indicadores
GET /api/dashboard/indicadores?tipoEntrega=RETIRADA&responsavel=atendente01
```

```
{
  "filtros": { "dataInicio": "2026-10-04", "dataFim": "2026-11-02", "tipoEntrega":
null, "status": null, "responsavel": null },
  "atualizadoEm": "2026-11-02T22:55:23",
  "dados": {
    "dataInicio": "2026-10-04", "dataFim": "2026-11-02",
    "faturamento": 10929.89, "totalPedidos": 112, "pedidosValidos": 102,
"ticketMedio": 107.16,
    "itensVendidos": 486, "totalDescontos": 108.11, "pedidosComDesconto": 11,
    "pedidosCancelados": 10, "taxaCancelamento": 8.9,
    "periodoAnterior": {
      "dataInicio": "2026-09-04", "dataFim": "2026-10-03",
      "faturamento": 6298.7, "totalPedidos": 68, "ticketMedio": 98.42,
      "variacaoFaturamento": 73.5, "variacaoPedidos": 64.7, "variacaoTicketMedio":
8.9
    }
  }
}
```

Erros: 400 data ou enum inválido 401 403 422 período invertido ou maior que 366 dias

6.5.2 GET /api/dashboard/vendas (RF02)

Evolução das vendas agrupada por período; grupos sem venda aparecem com zero.

Parâmetro extra	Tipo	Padrão	Valores
`agrupamento`	enum	`DIA`	`DIA` (rótulo “02/11”), `SEMANA` (“Sem. 26/10”, início na segunda), `MES` (“11/2026”)

```
GET /api/dashboard/vendas?agrupamento=SEMANA
```

```
"dados": [
  { "rotulo": "Sem. 28/09", "pedidos": 5,  "faturamento": 512.40 },
  { "rotulo": "Sem. 05/10", "pedidos": 22, "faturamento": 2287.10 },
  { "rotulo": "Sem. 12/10", "pedidos": 21, "faturamento": 2196.30 },
  ...
]
```

Erros: 400 agrupamento, data ou enum inválido 401 403 422

6.5.3 GET /api/dashboard/vendas-por-hora (RF02)

Pedidos e faturamento por hora do dia, da primeira à última hora com pedidos (lista vazia se não houver vendas).

```
"dados": [
  { "rotulo": "11h", "pedidos": 2,  "faturamento": 201.30 },
  { "rotulo": "12h", "pedidos": 8,  "faturamento": 861.70 },
  ...
  { "rotulo": "20h", "pedidos": 22, "faturamento": 2381.40 },
  { "rotulo": "22h", "pedidos": 11, "faturamento": 1190.60 }
]
```

Erros: 400 401 403 422

6.5.4 GET /api/dashboard/vendas-por-tipo (RF02)

Pedidos e faturamento por tipo de atendimento.

```
"dados": [
  { "rotulo": "Entrega",            "pedidos": 71, "faturamento": 7610.20 },
  { "rotulo": "Retirada no balcão", "pedidos": 31, "faturamento": 3319.69 }
]
```

Erros: 400 401 403 422

6.5.5 GET /api/dashboard/status (RF03)

Situação dos pedidos e tempos médios do fluxo. `porStatus` traz todos os status, inclusive os com zero; `emAndamento` são os que não estão FINALIZADO nem CANCELADO; os tempos estão em minutos (`null` se nenhum pedido chegou à etapa).

```
"dados": {
  "totalPedidos": 112, "emAndamento": 1, "finalizados": 101, "cancelados": 10,
  "porStatus": [
    { "status": "RECEBIDO", "descricao": "Recebido", "corBadge": "bg-info text-
dark", "icone": "bi-inbox", "quantidade": 0, "percentual": 0.0 },
    ...
    { "status": "SAIU_PARA_ENTREGA", "descricao": "Saiu para entrega", "corBadge":
"bg-secondary", "icone": "bi-scooter", "quantidade": 1, "percentual": 0.9 },
    { "status": "FINALIZADO", "descricao": "Finalizado", "corBadge": "bg-success",
"icone": "bi-flag-fill", "quantidade": 101, "percentual": 90.2 },
    { "status": "CANCELADO", "descricao": "Cancelado", "corBadge": "bg-danger",
"icone": "bi-x-octagon", "quantidade": 10, "percentual": 8.9 }
  ],
  "tempoMedioPreparoMinutos": 29.4, "tempoMedioAteSaidaMinutos": 36.9,
"tempoMedioTotalMinutos": 56.9
}
```

Erros: 400 401 403 422

6.5.6 GET /api/dashboard/produtos-mais-vendidos (RF04)

Ranking dos produtos dos pedidos não cancelados.

Parâmetro extra	Tipo	Padrão	Regra
`limite`	inteiro	`5`	de 1 a 50

Parâmetro extra	Tipo	Padrão	Regra
`ordenarPor`	enum	`QUANTIDADE`	`QUANTIDADE` ou `FATURAMENTO`

```
GET /api/dashboard/produtos-mais-vendidos?limite=3&ordenarPor=QUANTIDADE
```

```
"dados": [
  { "posicao": 1, "produtoId": "6730f1...", "nome": "Esfiha de Carne",
"quantidade": 234, "faturamento": 1521.00, "pedidos": 44,
"participacaoFaturamento": 13.8 },
  { "posicao": 2, "produtoId": "6730f1...", "nome": "Pizza Calabresa",
"quantidade": 66,  "faturamento": 3029.40, "pedidos": 44,
"participacaoFaturamento": 27.4 },
  { "posicao": 3, "produtoId": "6730f1...", "nome": "Pizza Quatro Queijos",
"quantidade": 64,  "faturamento": 3193.60, "pedidos": 43,
"participacaoFaturamento": 28.9 }
]
```

Erros: 400 limite não numérico ou ordenarPor inválido 401 403 422 limite fora de 1 a 50

6.5.7 GET /api/dashboard/pedidos-recentes (apoio)

Pedidos mais recentes do período, do mais novo para o mais antigo. Parâmetro extra: `limite` (inteiro, padrão `10`, de 1 a 50).

```
"dados": [
  {
    "id": "6730f2a9c1d4e5f6a7f72d98", "clienteNome": "Maria Demonstração",
    "tipoEntrega": "ENTREGA", "tipoEntregaDescricao": "Entrega",
    "status": "SAIU_PARA_ENTREGA", "statusDescricao": "Saiu para entrega",
"corBadge": "bg-secondary",
    "valorTotal": 158.30, "dataHoraEntrada": "2026-11-02T17:44:00",
"responsavelEntrada": "atendente01"
  }
]
```

Erros: 400 401 403 422

6.6 EXEMPLOS DE ERRO

Requisição	Status	Mensagem retornada
`/vendas?agrupamento=ANO`	400	Parâmetro 'agrupamento' com valor inválido: 'ANO'. Valores aceitos: DIA, SEMANA, MES.
`/indicadores?dataInicio=31/10/2026`	400	Parâmetro 'dataInicio' com valor inválido: '31/10/2026'. Use o formato AAAA-MM-DD (ex.: 2026-11-02).

Requisição	Status	Mensagem retornada
`/indicadores?status=XYZ`	400	Parâmetro 'status' com valor inválido: 'XYZ'. Valores aceitos: RECEBIDO, EM_PREPARACAO, PRONTO, SAIU_PARA_ENTREGA, RETIRADO, FINALIZADO, CANCELADO.
`/pedidos-recentes?limite=abc`	400	Parâmetro 'limite' com valor inválido: 'abc'. Informe um número inteiro.
`/indicadores?dataInicio=2026-11-10&dataFim=2026-11-01`	422	A data final não pode ser anterior à data inicial.
`/indicadores?dataInicio=2025-01-01&dataFim=2026-11-01`	422	O período máximo de consulta é de 366 dias.
`/produtos-mais-vendidos?limite=100`	422	O limite deve estar entre 1 e 50.
`/indicadores` como `atendente01`	403	Seu perfil não tem permissão para realizar esta operação.
`/indicadores` sem login	401	(corpo vazio)

7. DADOS DE DEMONSTRAÇÃO

Para os gráficos terem dados significativos, o `DemoDataInitializer` gera 180 pedidos fictícios nos últimos 60 dias, somente se a coleção de pedidos estiver vazia, nunca mistura com dados reais. Pode ser desligado em `application.properties` (`pediupatrao.demo.gerar-pedidos=false`).

Característica	Regra
Dias	Mais pedidos de sexta a domingo e leve crescimento nos dias recentes
Horários	Picos no almoço (12h a 13h) e no jantar (19h a 21h)
Itens	1 a 3 produtos; esfihas de 2 a 8 unidades, demais de 1 a 2
Atendimento	Cerca de 65% entrega e 35% retirada
Descontos	Cerca de 12% dos pedidos, de 5%, 10% ou 15%
Cancelamentos	Cerca de 8%, sempre antes da saída e com justificativa
Status	Pedidos antigos finalizados; os de hoje em etapas diferentes, conforme o horário
Reprodutibilidade	Semente fixa (`Random(42)`): os mesmos dados a cada geração

8. EVIDÊNCIAS DE TESTES

8.1 TESTES AUTOMATIZADOS

Execução no NetBeans (Test, Alt+F6): 61 testes, 0 falhas, 0 erros, sendo 37 da atividade anterior e 24 do dashboard.

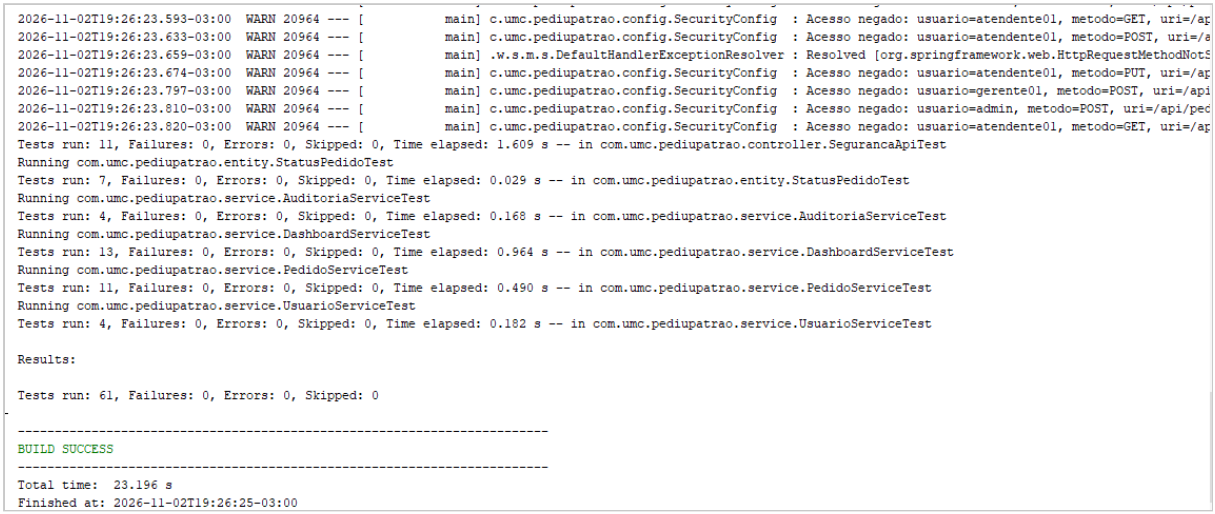


Figura 1: Resultado da execução dos testes automatizados (Maven Surefire)

8.1.1 DashboardServiceTest: cálculos (MongoDB simulado com Mockito; “hoje” fixado em 02/11/2026)

#	Teste	RF	Resultado
1	Sem datas informadas, usa os últimos 30 dias até hoje	RF05	Passou
2	Data final anterior à inicial é recusada antes de consultar o banco	RF05	Passou
3	Período maior que 366 dias é recusado	RF05	Passou
4	Período, tipo de entrega, status e responsável são aplicados na consulta ao MongoDB	RF05	Passou
5	Limite fora do intervalo de 1 a 50 é recusado	RF04	Passou
6	Faturamento ignora cancelados; calcula ticket médio, itens, descontos, cancelamentos e variação	RF01	Passou
7	Período sem pedidos não divide por zero e a variação fica vazia	RF01	Passou
8	Vendas por dia incluem os dias sem venda com zero	RF02	Passou
9	Vendas por semana agrupam de segunda a domingo e ignoram cancelados	RF02	Passou
10	Vendas por hora vão da primeira à última hora com pedidos	RF02	Passou
11	Conta pedidos por status, em andamento e calcula os tempos médios	RF03	Passou

#	Teste	RF	Resultado
12	Ranking por quantidade e por faturamento, sem contar pedidos cancelados	RF04	Passou
13	Pedidos recentes: consulta ordenada do mais novo para o mais antigo, com limite		Passou

8.1.2 DashboardApiTest: camada web com o SecurityConfig real

#	Teste	Esperado	Resultado
1	Dashboard sem login	401	Passou
2	ATENDENTE não acessa o dashboard	403	Passou
3	GERENTE acessa os indicadores no formato filtros/atualizadoEm/dados	200	Passou
4	ADMIN acessa o dashboard	200	Passou
5	Os filtros da URL chegam ao service convertidos	200 + filtros corretos	Passou
6	Limite e critério do ranking chegam ao service	200	Passou
7	Agrupamento inexistente	400 com valores aceitos	Passou
8	Data em formato errado	400	Passou
9	Status inexistente	400 com valores aceitos	Passou
10	Limite não numérico	400	Passou
11	Regra de negócio violada (data final antes da inicial)	422	Passou

8.2 TESTES MANUAIS DA TELA E DA API

#	Cenário	Passos	Resultado esperado	Result.	Fig.
M1	Dashboard completo	Login `gerente01` → menu Dashboard	Cards, gráficos, ranking e pedidos recentes preenchidos; período “30 dias · até hoje”	OK	2 a 5
M2	Filtro rápido	Clicar em Hoje	Período “1 dia · até hoje” e números do dia	OK	6

#	Cenário	Passos	Resultado esperado	Result.	Fig.
M3	Filtro por atendimento	Atendimento = Retirada → Filtrar	Etiqueta “Atendimento: Retirada no balcão”	OK	7
M4	Filtro inválido	Data final antes da inicial → Filtrar	Aviso amarelo “Filtro inválido”	OK	8
M5	Servidor fora do ar	Parar o sistema → Atualizar	Aviso vermelho; últimos dados mantidos	OK	9
M6	Perfil sem acesso	Login `atendente01` → `/dashboard`	Link oculto no menu; página 403	OK	10
M7	Atualização automática	Aguardar 60 s	“Última atualização” muda sozinha	OK	
M8	API 200	`GET /api/dashboard/indicadores`	JSON no envelope padrão	OK	11
M9	API 400	`GET /api/dashboard/vendas?agrupamento=ANO`	400 com valores aceitos	OK	12
M10	API 422	`/indicadores?dataInicio=2026-11-10&dataFim=2026-11-01`	422 com a mensagem da regra	OK	13
M11	API 403	`/indicadores` como `atendente01`	403 “Acesso negado”	OK	14

8.3 IMAGENS

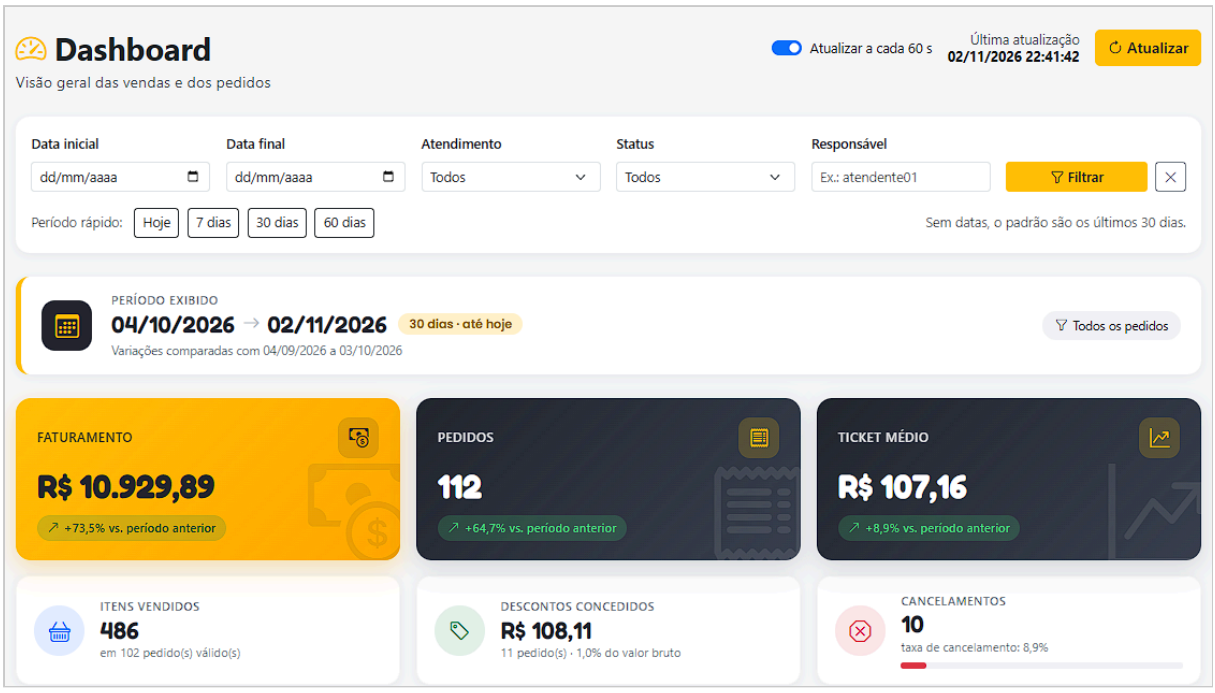


Figura 2: Dashboard: filtros, período exibido em destaque e indicadores (RF01, RF05)

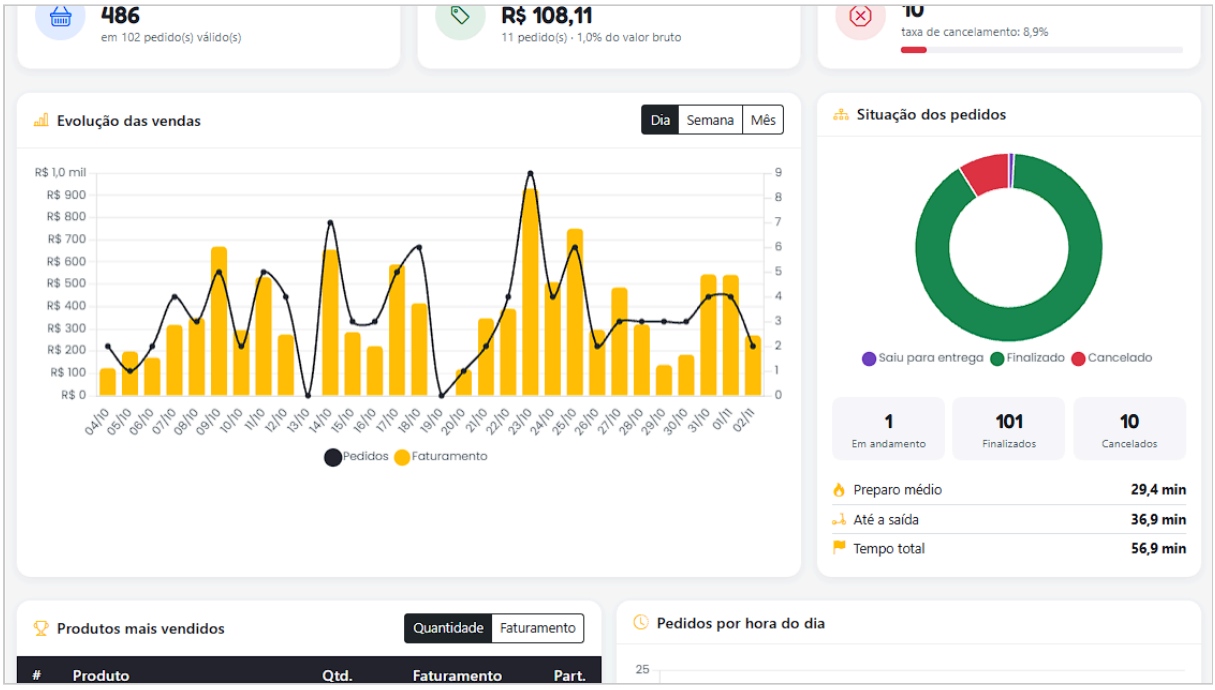


Figura 3: Evolução das vendas por dia e situação dos pedidos com tempos médios (RF02, RF03)

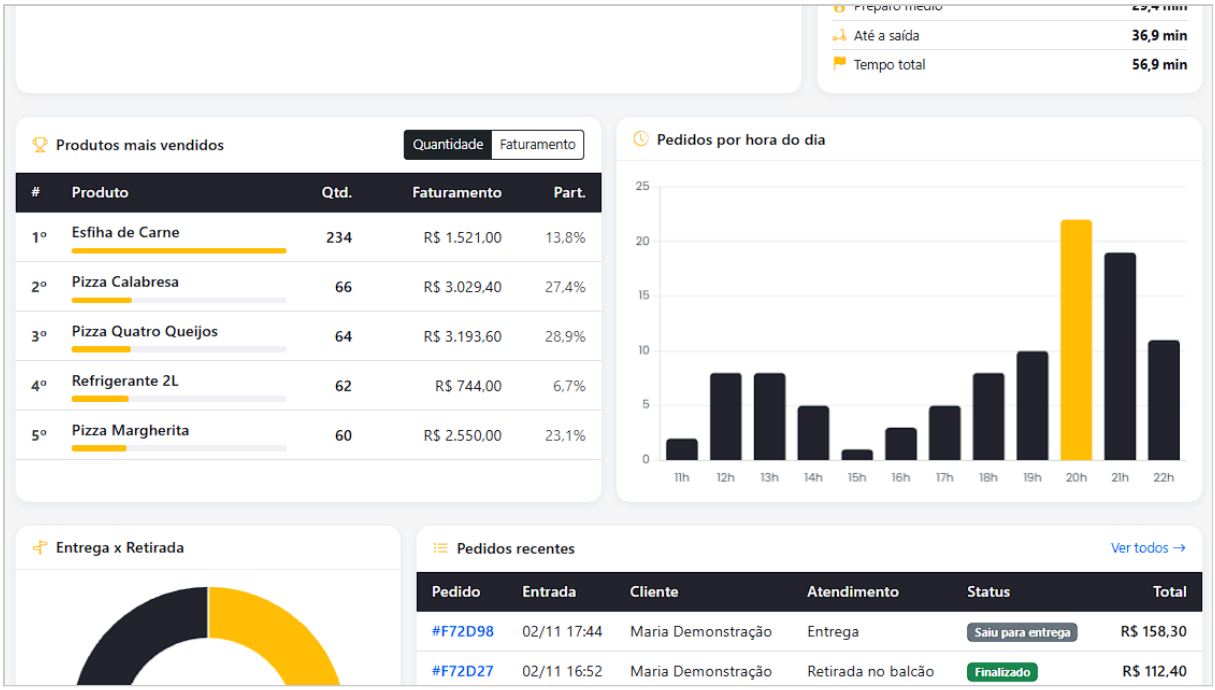


Figura 4: Produtos mais vendidos e pedidos por hora, com o pico em destaque (RF02, RF04)

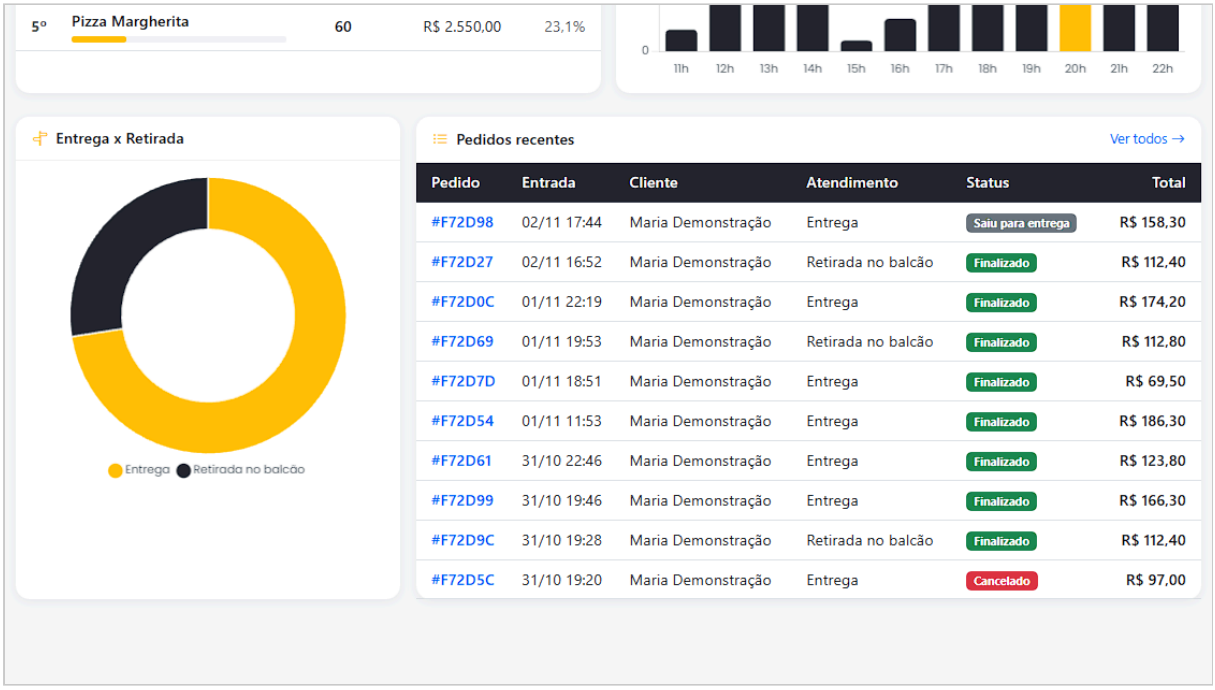


Figura 5: Entrega x retirada e pedidos recentes (RF02)

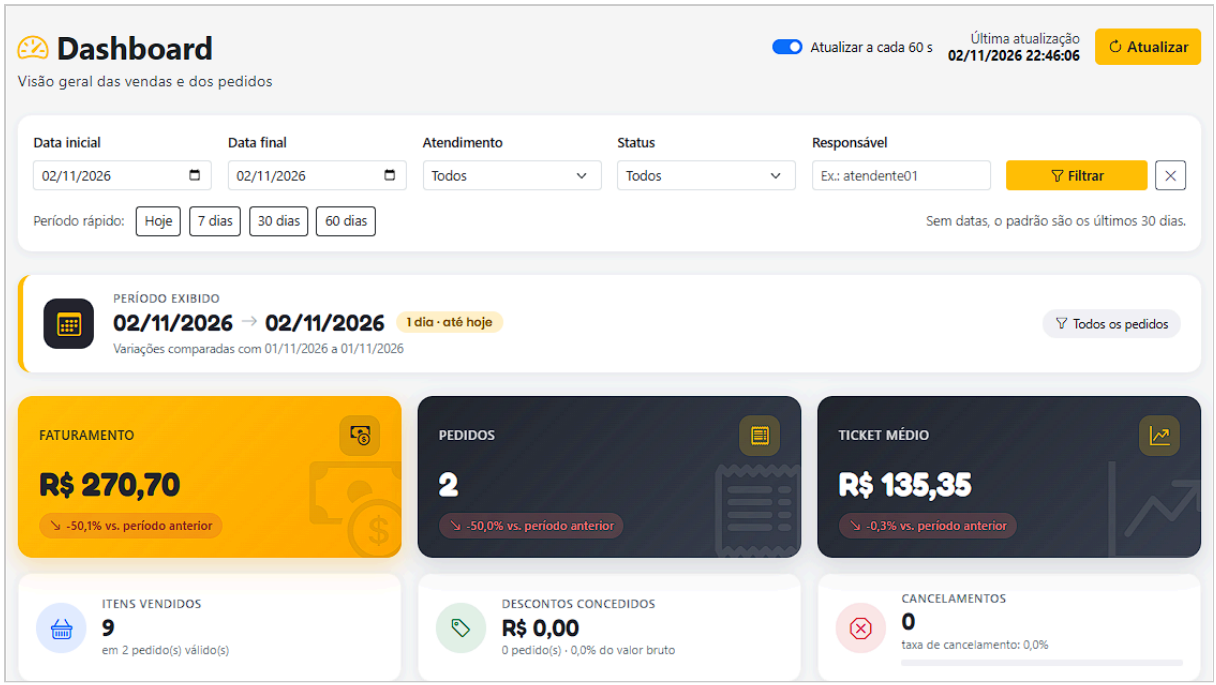


Figura 6: Filtro rápido “Hoje”, com variação em relação ao dia anterior (RF05)

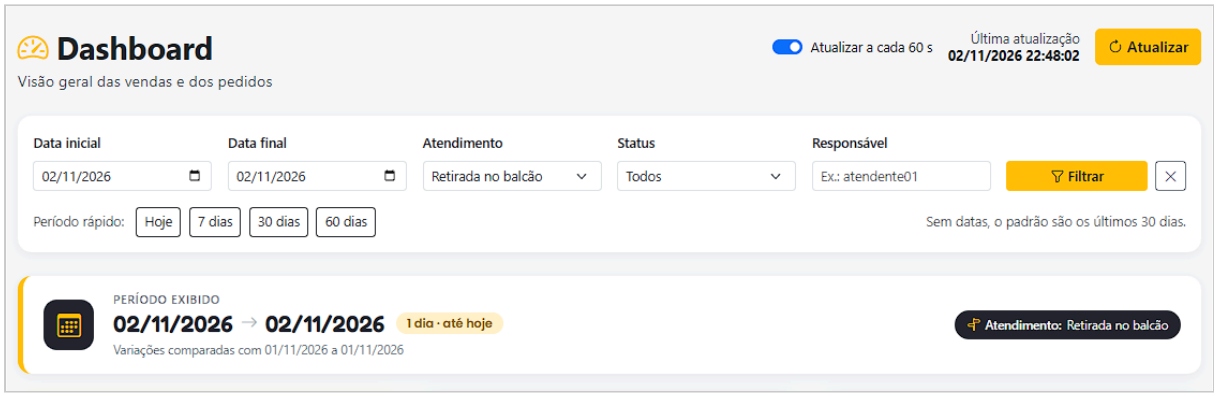


Figura 7: Filtro por atendimento, exibido como etiqueta na faixa do período (RF05)

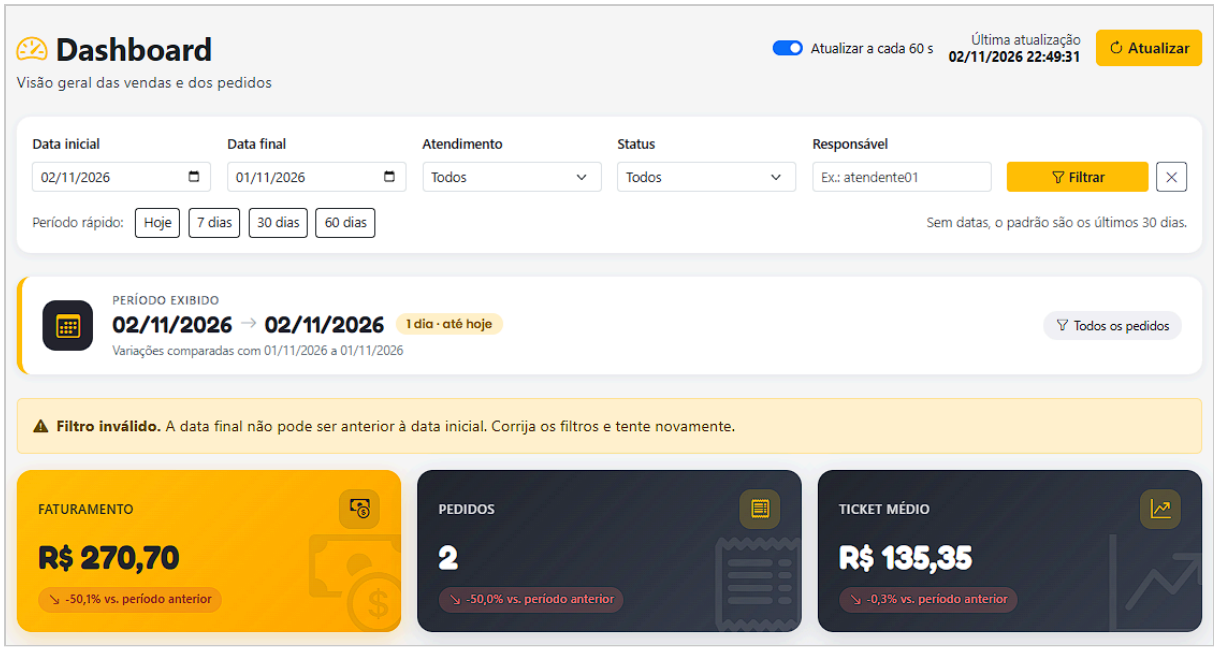


Figura 8: Aviso de filtro inválido: data final anterior à inicial (RF07)

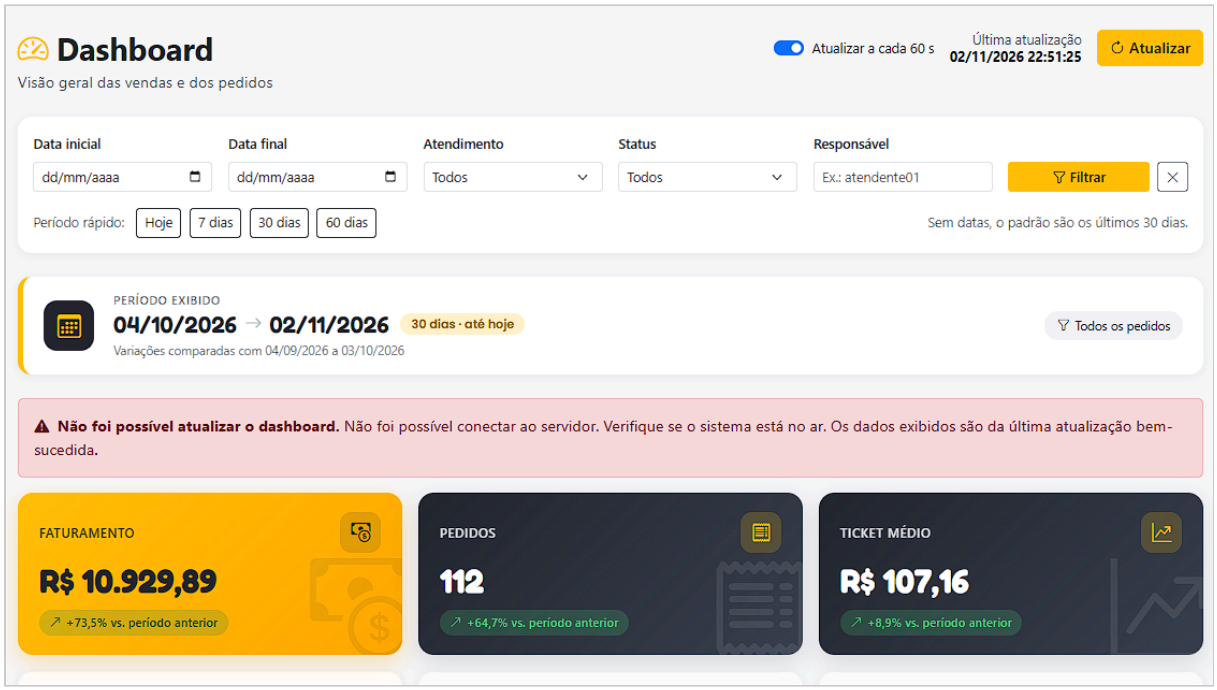


Figura 9: Servidor fora do ar: aviso de erro com os últimos dados mantidos na tela (RF07)

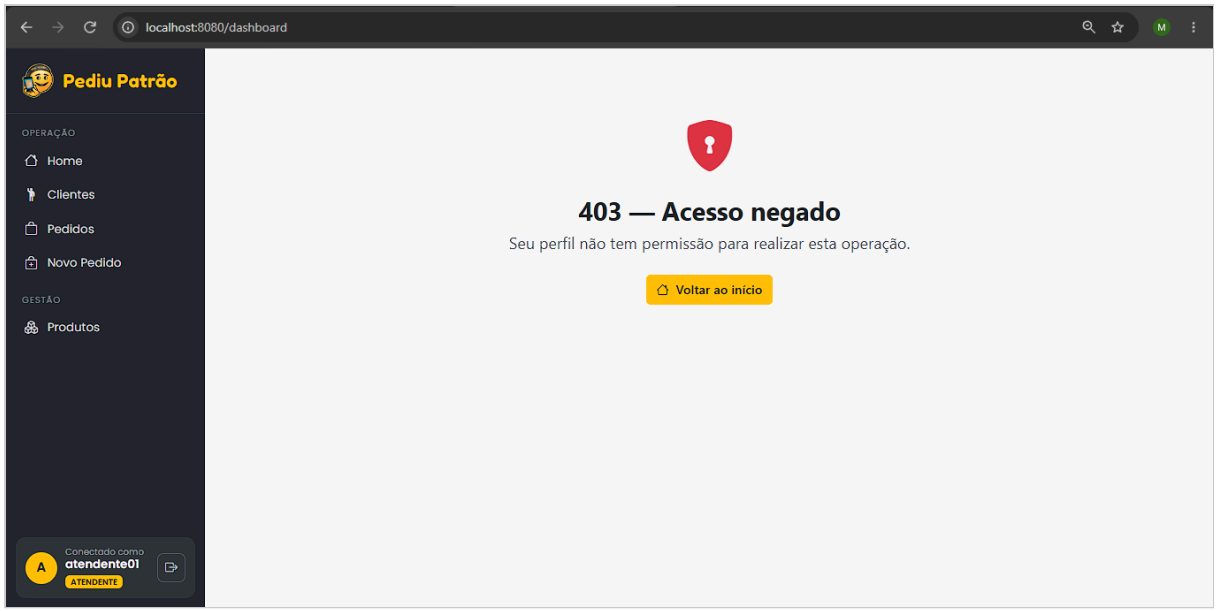
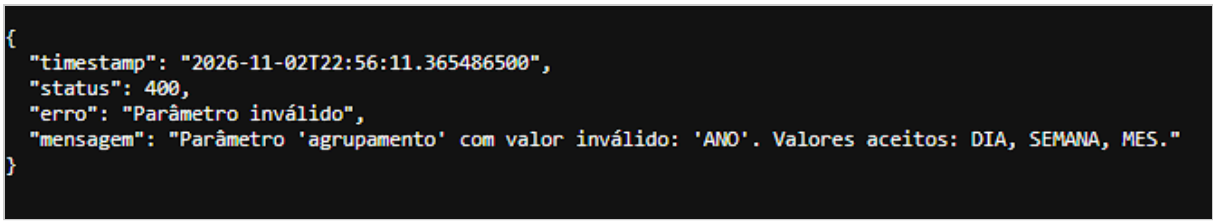


Figura 10: ATENDENTE acessando /dashboard: link oculto no menu e página 403 (RNF01)

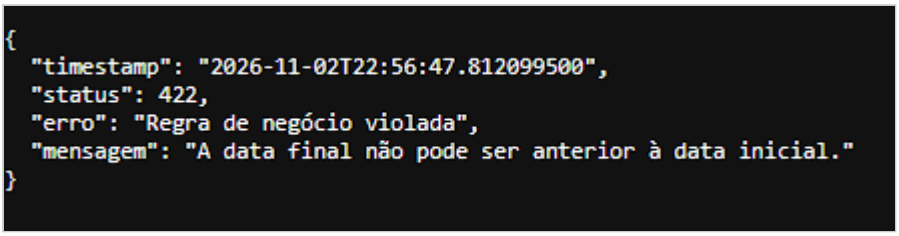

```
{
  "filtros": {
    "dataInicio": "2026-10-04",
    "dataFim": "2026-11-02",
    "tipoEntrega": null,
    "status": null,
    "responsavel": null
  },
  "atualizadoEm": "2026-11-02T22:55:23",
  "dados": {
    "dataInicio": "2026-10-04",
    "dataFim": "2026-11-02",
    "faturamento": 10929.89,
    "totalPedidos": 112,
    "pedidosValidos": 102,
    "ticketMedio": 107.16,
    "itensVendidos": 486,
    "totalDescontos": 108.11,
    "pedidosComDesconto": 11,
    "pedidosCancelados": 10,
    "taxaCancelamento": 8.9,
    "periodoAnterior": {
      "dataInicio": "2026-09-04",
      "dataFim": "2026-10-03",
      "faturamento": 6298.7,
      "totalPedidos": 68,
      "ticketMedio": 98.42,
      "variacaoFaturamento": 73.5,
      "variacaoPedidos": 64.7,
      "variacaoTicketMedio": 8.9
    }
  }
}
```


Figura 11: API: 200 OK em /indicadores (RF06)



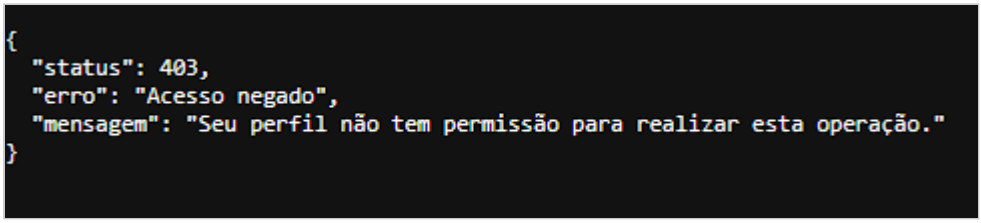
```
{
  "timestamp": "2026-11-02T22:56:11.365486500",
  "status": 400,
  "erro": "Parâmetro inválido",
  "mensagem": "Parâmetro 'agrupamento' com valor inválido: 'ANO'. Valores aceitos: DIA, SEMANA, MES."
}
```


Figura 12: API: 400 com agrupamento inválido



```
{
  "timestamp": "2026-11-02T22:56:47.812099500",
  "status": 422,
  "erro": "Regra de negócio violada",
  "mensagem": "A data final não pode ser anterior à data inicial."
}
```


Figura 13: API: 422 com período invertido



```
{
  "status": 403,
  "erro": "Acesso negado",
  "mensagem": "Seu perfil não tem permissão para realizar esta operação."
}
```


Figura 14: API: 403 para o ATENDENTE

9. GITFLOW E VERSIONAMENTO

Branch	Finalidade
`main`	Versão estável entregue
`develop`	Integração das funcionalidades
`feature/api-dashboard`	API: dados de demonstração, service, endpoints e testes
`feature/dashboard-tela`	Tela do dashboard
`feature/documentacao`	Esta documentação
`release/2.0.0`	Preparação da versão entregue (merge na `main` e tag `v2.0.0`)

10. COMO EXECUTAR

1. Ter o Java 21 e o MongoDB rodando em `localhost:27017`.
2. Abrir o projeto no NetBeans e executar (F6) ou, no terminal, `mvnw spring-boot:run`.

3. Acessar `http://localhost:8080` e entrar com um dos usuários abaixo.
4. Abrir o menu Gestão > Dashboard.
5. Na primeira execução com a coleção `pedidos` vazia, são gerados 180 pedidos de demonstração.
6. Testes automatizados: botão direito no projeto → Test (Alt+F6) ou `mvnw test`.

Usuário	Senha	Perfil	Dashboard
`admin`	`teste123`	ADMIN	Acessa
`gerente01`	`gerente123`	GERENTE	Acessa
`atendente01`	`atendente123`	ATENDENTE	Sem acesso